

Real-world (act out a canary release) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below use this shared setup — 20 fake test requests, and a `run_eval_suite` stub where `"v2"` 's prompt scores better than `"v1"` 's:

```
import random

prompts = {
    "v1": "Answer the customer's question.",
    "v2": "Answer the customer's question politely and cite the source.",
}
test_requests = list(range(20)) # 20 stand-in requests

def run_eval_suite(prompt_text):
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}
```

Each example below calls `random.seed(...)` with a fixed number so the "random" routing is the same, reproducible split every time you run it — remove the seed line for genuinely random routing in real use.

Basic Version

Approach 1 — the direct way

```
random.seed(0)

def route_and_score(test_requests, prompts):
    results = []
    for request in test_requests:
        if random.random() < 0.2:
            version = "v2"
        else:
            version = "v1"
        score = run_eval_suite(prompts[version])["score"]
        results.append({"version": version, "score": score})
```

```
    return results

def average_by_version(results, version):
    matching = []
    for r in results:
        if r["version"] == version:
            matching.append(r["score"])
    return sum(matching) / len(matching)

results = route_and_score(test_requests, prompts)
print("v1 avg:", average_by_version(results, "v1"))
print("v2 avg:", average_by_version(results, "v2"))
```

Expected output:

```
v1 avg: 0.6
v2 avg: 1.0
```

This version works correctly for what the exercise asks. It's missing type hints, doesn't track cost, and would crash with a `ZeroDivisionError` if a run happened to route zero requests to `"v2"` — all fine for a first working version, all fixed below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a `canary_share` parameter, cost tracking, and a `summarize()` function

```
random.seed(1)

def route_and_score(test_requests: list, prompts: dict, canary_share: float = 0.2) -> list[dict]:
    results = []
    for request in test_requests:
        if random.random() < canary_share:
            version = "v2"
        else:
            version = "v1"
        eval_result = run_eval_suite(prompts[version])
        results.append({
            "version": version,
            "score": eval_result["score"],
            "cost": len(prompts[version]) * 0.0001,
        })
    return results

def summarize(results: list[dict], version: str) -> dict:
    matching = []
    for r in results:
        if r["version"] == version:
            matching.append(r)

    scores = []
    costs = []
    for r in matching:
        scores.append(r["score"])
        costs.append(r["cost"])

    return {
        "version": version,
        "count": len(matching),
        "avg_score": sum(scores) / len(scores),
        "avg_cost": sum(costs) / len(costs),
    }

results = route_and_score(test_requests, prompts)
```

```
control = summarize(results, "v1")
canary = summarize(results, "v2")
print(control)
print(canary)
```

Expected output:

```
{'version': 'v1', 'count': 15, 'avg_score': 0.6, 'avg_cost': 0.0031}
{'version': 'v2', 'count': 5, 'avg_score': 1.0, 'avg_cost': 0.006}
```

(the real printed float carries extra trailing digits from binary floating point, like
`0.0030999999999999995` — rounded here for readability)

Difference from Basic: Approach 1 adds a `canary_share` parameter instead of hardcoding `0.2` inline, tracks a per-request cost estimate alongside score, and replaces `average_by_version` with `summarize()`, which returns a full dict (`count` , `avg_score` , `avg_cost`) instead of just one number — `count` in particular matters, since it's what Advanced's sample-size guard checks.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — an applied promotion rule, with a sample-size guard

```
def evaluate_canary(control_summary: dict, canary_summary: dict, min_sample_size:
    if canary_summary["count"] < min_sample_size:
        return "insufficient data"

    score_gap = canary_summary["avg_score"] - control_summary["avg_score"]
    cost_ratio = canary_summary["avg_cost"] / control_summary["avg_cost"]

    if score_gap >= 0 and cost_ratio <= 1.2:
        return "promote"
    return "hold"

random.seed(1)
results = route_and_score(test_requests, prompts)
control = summarize(results, "v1")
canary = summarize(results, "v2")

verdict = evaluate_canary(control, canary)
score_gap = canary["avg_score"] - control["avg_score"]
cost_ratio = canary["avg_cost"] / control["avg_cost"]
print(f"verdict: {verdict} (score gap {score_gap:+.2f}, cost ratio {cost_ratio:.2f})")
```

Expected output:

```
verdict: hold (score gap +0.40, cost ratio 1.94x, canary n=5)
```

The canary group (v2) scores strictly better (+0.40) — on quality alone it looks like a clear win. But it also costs nearly twice as much per request (1.94x), which fails the "no more than 20% higher" half of the rule, so `evaluate_canary` correctly holds rather than promoting. This is exactly the situation the rule exists to catch: a human skimming just the score column would promote a version that's secretly almost 2x more expensive.

Approach 2 — a stricter rule, and a realistic small-sample "insufficient data" result

```
def evaluate_canary(control_summary: dict, canary_summary: dict, min_sample_size: int):
    if canary_summary["count"] < min_sample_size:
        return "insufficient data"

    score_gap = canary_summary["avg_score"] - control_summary["avg_score"]
    cost_ratio = canary_summary["avg_cost"] / control_summary["avg_cost"]

    if score_gap >= 0 and cost_ratio <= 1.2:
        return "promote"
    if score_gap < 0:
        return "hold – worse quality"
    return "hold – quality ok, cost too high"

random.seed(1)
small_requests = list(range(4)) # too few requests for a trustworthy canary group
results = route_and_score(small_requests, prompts)
control = summarize(results, "v1")
canary = summarize(results, "v2")

print(f"canary n={canary['count']} -> {evaluate_canary(control, canary)}")
```

Expected output:

```
canary n=1 -> insufficient data
```

With only 4 requests total and a 20% canary share, the canary group lands at just 1 request — nowhere near the `min_sample_size=5` default. Exactly the situation Hint 1's Advanced section warns about: too few canary requests to trust an average, so the function says so explicitly instead of confidently promoting or holding on a single data point.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate computes and prints both groups' numbers, but leaves the actual "is this safe" judgment to whoever reads the printout. Approach 1 turns that into an applied, callable decision with an explicit rule and named numbers in the printed verdict. Approach 2 keeps the same rule but reports *why* a hold happened (worse quality vs. too expensive are different problems needing different fixes), and demonstrates the sample-size guard actually firing on a too-small canary group — the exact case a real rollout must not silently ignore.

Which one should you actually write? Approach 2's shape — a named rule, a sample-size guard, and a verdict that explains itself — is what you want in real use; a bare `"promote" / "hold"` (Approach 1) is fine for a quick script but forces the reader to go recompute the gap and ratio themselves to understand *why*. In your real Project 5 canary rollout, replace the stand-in cost

estimate (`len(prompt) * 0.0001`) with your actual per-request token cost from your observability setup, and replace `run_eval_suite` 's fake scores with your real Doc13 test suite call.